



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 2 – POO Y DIAGRAMAS DE CLASE



EL PARADIGMA ORIENTADO A OBJETOS

- Paradigma que utiliza el concepto de objeto para estructurar el software
- Un objeto representa una entidad de la vida real, posee:
 - Estado
 - Comportamiento
- Un sistema logra su propósito a través de la interacción que se da entre los objetos

EL PARADIGMA ORIENTADO A OBJETOS: PRINCIPIOS

- **Encapsulamiento:** protección de datos y ocultamiento de información
- **Herencia:** reutilización y especialización de objetos
- **Polimorfismo:** flexibilidad y compatibilidad de objetos
- **Abstracción:** simplificación de conceptos, estado y comportamiento relevantes para el modelo

EL PARADIGMA ORIENTADO A OBJETOS: ENCAPSULAMIENTO

- La información de un objeto está oculta del exterior
- Atributos (estado) con acceso privado, métodos (comportamiento) con acceso público
- La modificación del estado se hace a través de sus métodos

```
package ar.edu.uade.adoo.intro;

public class Vehiculo {
    2 usages
    private int velocidadActual;
    3 usages
    private int velocidadMaxima;

    public int getVelocidadMaxima() {
        return velocidadMaxima;
    }

    public void setVelocidadMaxima(int velocidadMaxima) {
        this.velocidadMaxima = velocidadMaxima;
    }

    public void acelerar(int velocidad) {
        if (velocidadActual + velocidad < velocidadMaxima) {
            velocidadActual += velocidad;
        }
    }
}
```

EL PARADIGMA ORIENTADO A OBJETOS: HERENCIA

- Se comparten métodos y datos entre clases y subclases
- Una nueva clase (subclase) puede utilizar la implementación de la superclase (clase base) o especificar una nueva

```
package ar.edu.uade.adoo.intro;

public class Moto extends Vehiculo {
    2 usages
    private int angulo;

    public int getAngulo() {
        return angulo;
    }

    public void wheelie(int velocidad) {
        acelerar(velocidad);
        cambiarAngulo(45);
    }

    1 usage
    public void cambiarAngulo(int angulo) {
        this.angulo = angulo;
    }
}
```

```
package ar.edu.uade.adoo.intro;

public class Auto extends Vehiculo {
    2 usages
    private boolean radioEncendida;

    public void apagarRadio() {
        if (radioEncendida) {
            radioEncendida = false;
        }
    }
}
```

EL PARADIGMA ORIENTADO A OBJETOS: HERENCIA

- Podemos impedir que se instancia la clase Vehiculo agregando el modificador “abstract”
- Podemos obligar a las clases derivadas o subclasses a que implementen un método agregando el modificador “abstract”

```
package ar.edu.uade.adoo.intro;

public abstract class Vehiculo {
    2 usages
    private int velocidadActual;
    3 usages
    private int velocidadMaxima;

    2 implementations
    public abstract void encender();

    public int getVelocidadMaxima() {
        return velocidadMaxima;
    }

    public void setVelocidadMaxima(int velocidadMaxima) {
        this.velocidadMaxima = velocidadMaxima;
    }

    1 usage
    public void acelerar(int velocidad) {
        if (velocidadActual + velocidad < velocidadMaxima) {
            velocidadActual += velocidad;
        }
    }
}
```

EL PARADIGMA ORIENTADO A OBJETOS: POLIMORFISMO

- Un objeto tiene distintos comportamientos
- Tipos:
 - Polimorfismo Ad Hoc (sobrecarga de métodos, muchas definiciones para misma función)
 - Polimorfismo paramétrico (los datos de una clase se pueden manipular sin importar el tipo)
 - Polimorfismo de subtipos (una clase derivada sustituye el comportamiento de una clase base)

```
package ar.edu.uade.adoo.intro;

public class Auto extends Vehiculo {
    3 usages
    private boolean radioEncendida;

    public void apagarRadio() {
        if (radioEncendida) {
            radioEncendida = false;
        }
    }

    @Override
    public void encender() {
        System.out.println("Enciendo auto");
    }

    public void encender(boolean prenderRadio) {
        System.out.println("Enciendo auto");
        if (prenderRadio) {
            radioEncendida = true;
        }
    }
}
```

```
package ar.edu.uade.adoo.intro;

public class Moto extends Vehiculo {
    2 usages
    private int angulo;

    public int getAngulo() {
        return angulo;
    }

    public void wheelie(int velocidad) {
        acelerar(velocidad);
        cambiarAngulo(45);
    }

    1 usage
    public void cambiarAngulo(int angulo) {
        this.angulo = angulo;
    }

    @Override
    public void encender() {
        System.out.println("Enciendo moto");
    }
}
```

EL PARADIGMA ORIENTADO A OBJETOS: ABSTRACCIÓN

- Es un principio que permite simplificar conceptos y comportamiento complejos a través de modelos o representaciones abstractas y simplificadas

```
package ar.edu.uade.adoo.intro;

public abstract class Vehiculo {
    2 usages
    private int velocidadActual;
    3 usages
    private int velocidadMaxima;

    2 implementations
    public abstract void encender();

    public int getVelocidadMaxima() {
        return velocidadMaxima;
    }

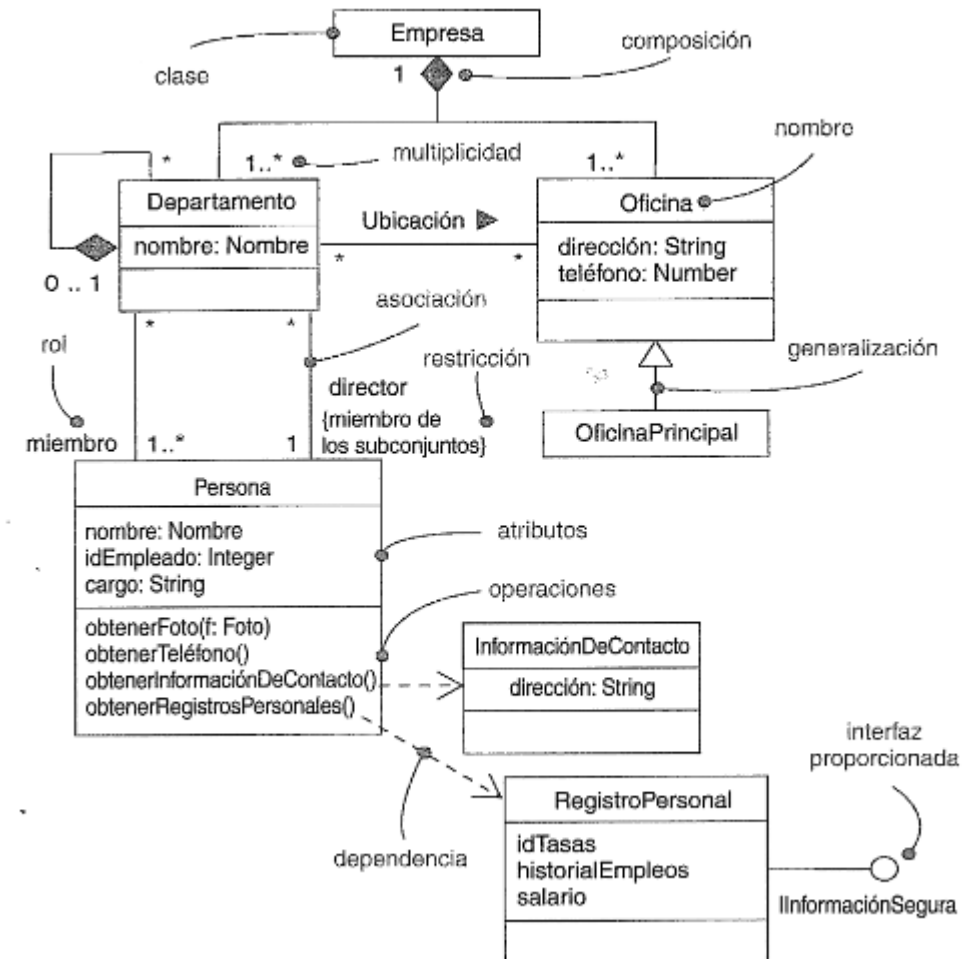
    public void setVelocidadMaxima(int velocidadMaxima) {
        this.velocidadMaxima = velocidadMaxima;
    }

    1 usage
    public void acelerar(int velocidad) {
        if (velocidadActual + velocidad < velocidadMaxima) {
            velocidadActual += velocidad;
        }
    }
}
```


DIAGRAMAS DE CLASE

INTRODUCCIÓN

- Los diagramas de clase contienen:
 - Clases
 - Interfaces
 - Relaciones de dependencia, generalización y asociación



DIAGRAMAS DE CLASE

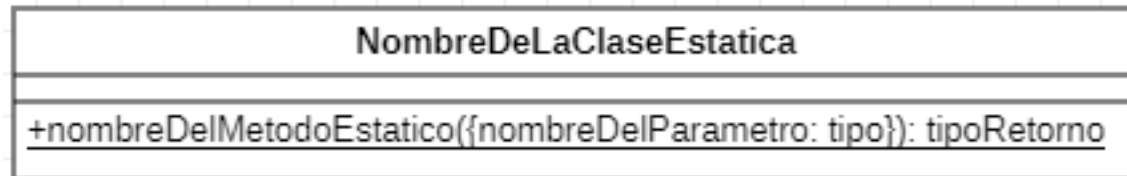
CLASES

NombreDeLaClase
-nombreDelAtributo: tipo = valorInicial
+nombreDelMetodo({nombreDelParametro: tipo}): tipoRetorno

- Tipos de visibilidad:
 - + atributos u operaciones públicas
 - - atributos u operaciones privadas
 - # atributos u operaciones protegidas

DIAGRAMAS DE CLASE

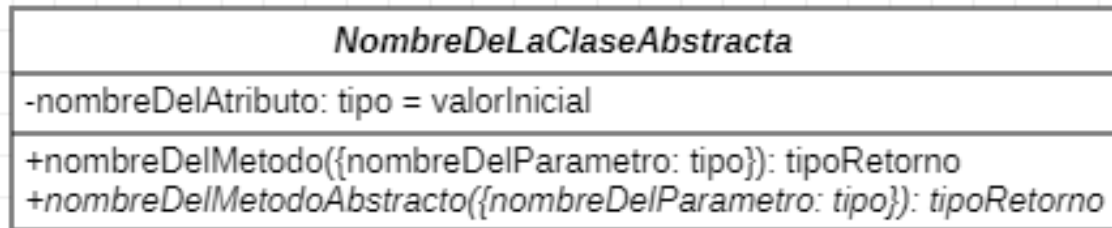
CLASES ESTÁTICAS



- Las clases estáticas no necesitan instanciarse
- Útiles cuando no tienen que almacenar información (por ejemplo, realizar cálculos)

DIAGRAMAS DE CLASE

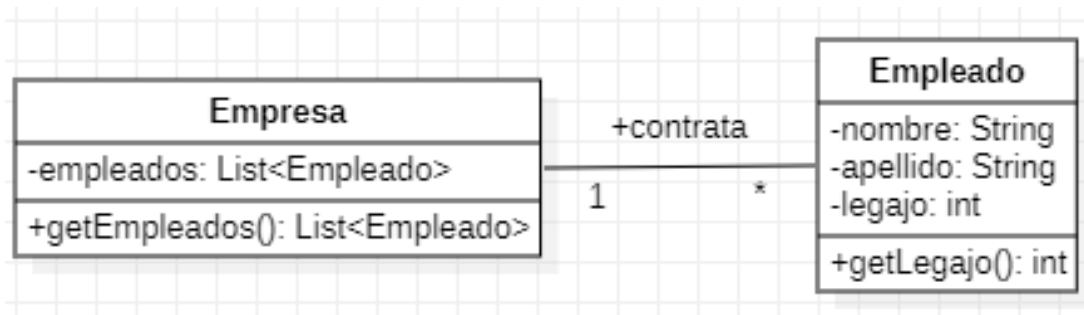
CLASES ABSTRACTAS



- No pueden ser instanciadas
- Si una clase tiene métodos abstractos declarando su existencia pero no su implementación, entonces también es abstracta
- Una clase abstracta puede tener métodos no abstractos

DIAGRAMAS DE CLASE

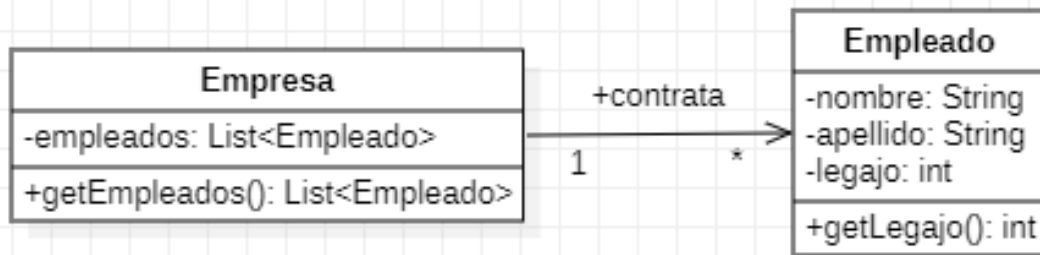
RELACIONES



- Una relación representa una dependencia entre clases distintas o una misma (dependencia reflexiva)
- Poseen:
 - Multiplicidad: número de elementos de una clase que participan. Se puede indicar un número, un rango ... (n o $*$ para identificar un número cualquiera)
 - Nombre: para entender la relación, suelen usarse verbos

DIAGRAMAS DE CLASE

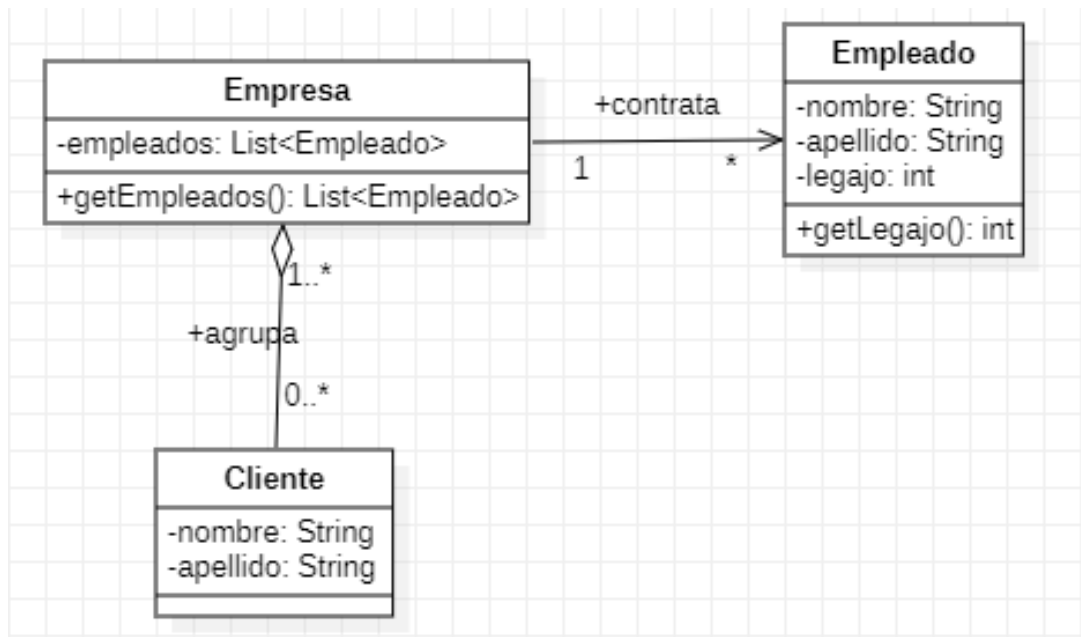
RELACIONES: ASOCIACIÓN



- La asociación indica la conexión entre clases
- La navegación (flechas) indican cómo debe recorrerse la relación

DIAGRAMAS DE CLASE

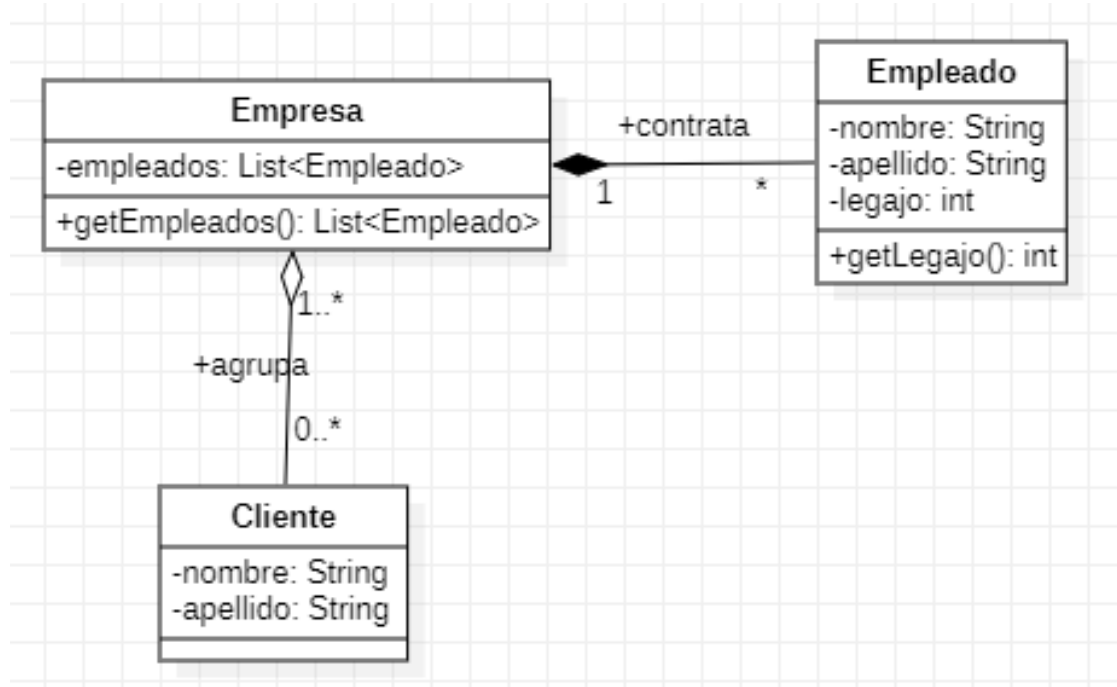
RELACIONES: AGREGACIÓN



- Es un tipo especial de asociación que representa una relación “es parte de”
- Las clases de la relación de agregación tienen tiempos de vida separados (su existencia es independiente)

DIAGRAMAS DE CLASE

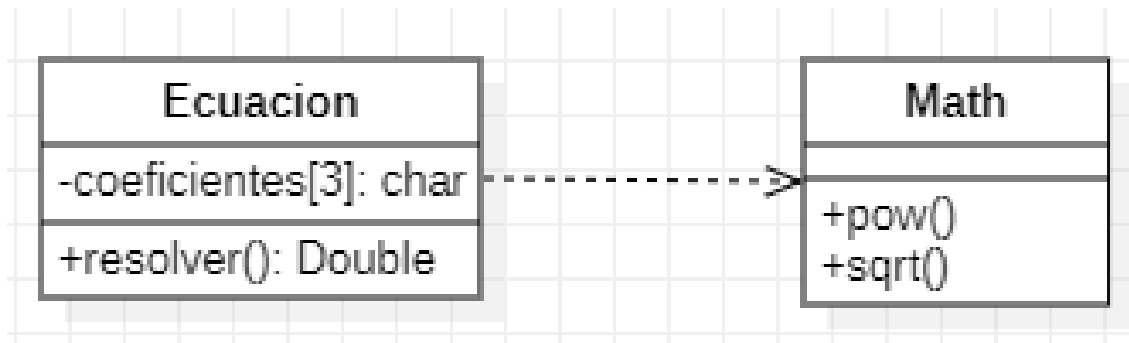
RELACIONES: COMPOSICIÓN



- Es una relación más fuerte en donde las partes se destruyen cuando el todo se destruye
- Es una agregación con una fuerte relación de pertenencia y vidas coincidentes de la parte con el todo

DIAGRAMAS DE CLASE

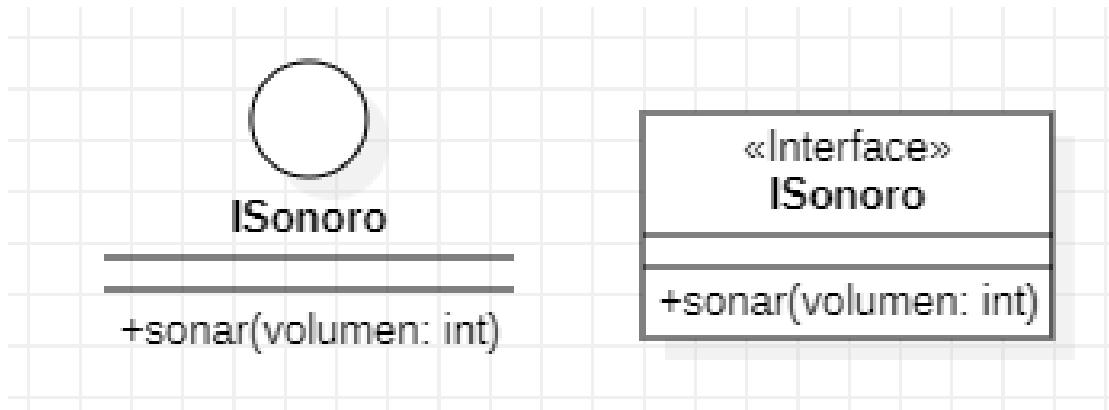
RELACIONES: DEPENDENCIA



- Es una relación que representa que una clase depende de otra para que pueda llevar a cabo su funcionalidad

DIAGRAMAS DE CLASE

INTERFACES



- Es un contrato que indica atributos y funciones que obligatoriamente deben implementarse por parte de una instancia asociada

CLASES ABSTRACTAS VS INTERFACES

Clase abstracta	Interfaz
Puede tener cualquier tipo de variables de instancia o estáticas, mutables o inmutables	Puede tener variables estáticas finales únicamente, nunca puede cambiar su propio estado
Una clase puede heredar una única clase abstracta	Una clase puede implementar múltiples interfaces
Puede ser heredada o extender cualquier clase (abstracta o no)	Puede implementar y extender otras interfaces
Pueden tener constructor	No pueden tener constructor
Puede tener métodos abstractos públicos o protegidos, además de métodos no abstractos	Solo puede tener métodos públicos (abstractos)

BIBLIOGRAFÍA

- BOOCH, G. JACOBSON, I. RUMBAUGH, J. El lenguaje unificado de modelado. Addison-Wesley Iberoamerica. Edición 2000. ISBN 8478290281. Capítulos 5 y 10.



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 2 – POO Y DIAGRAMAS DE CLASE

